How we built it

A plain-words guide to the baby guide and message board

Written 6 November 2026 for a product manager who wants to understand web building well enough to do it next time without following a step-by-step guide.

1. The result in one minute

We built a small website with two parts. /guide is a practical baby-care guide: a fixed page, the same for everyone. The home page, /, is a public message board: visitors write their name and a kind message, and the newest messages appear on top.

It works on phones, has a dark mode, and updates without reloading the page. Everything is kept as code on GitHub and goes live automatically whenever we push a change.

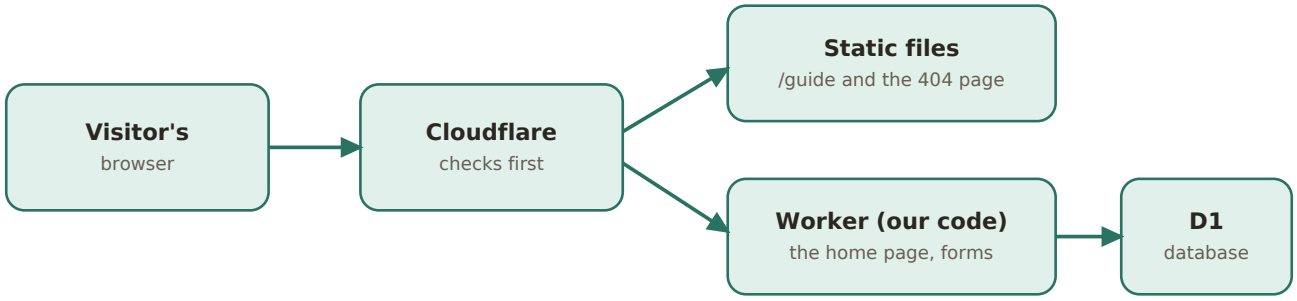
2. The mental model: how anything on the web works

Every web page is a conversation. The **browser** (a visitor's phone or laptop) asks for something, and a computer on the internet answers. That is all of it. Almost every project, small or large, is built from three layers:

Layer	Its job	What we used
Front end (what people see)	Looks and interaction. Built from HTML (structure), CSS (looks) and JavaScript (behavior).	The pages themselves, and the small script that makes the form feel instant.
Back end (the logic)	Decides what to answer, checks what people send, talks to the database.	A **Worker**: a small program that Cloudflare runs on demand each time someone visits.
Data (the memory)	Remembers things between visits.	**D1**, Cloudflare's database. It holds the messages.

We never rented or looked after a server. Cloudflare runs our code only when someone visits. That is what "serverless" means: you write the logic, they run it.

The path of one visit. If a file exists for the address, Cloudflare serves it directly. Otherwise our Worker runs, and it asks the database only when it needs to remember or recall something.

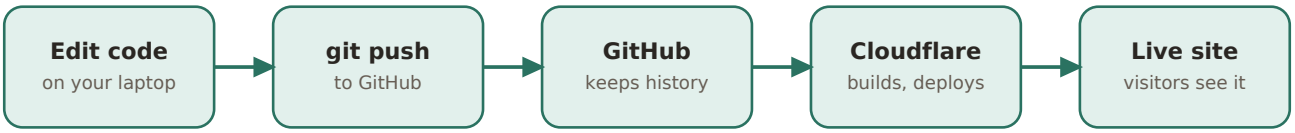


3. The first product decision: static or dynamic?

- **Static** means a file served exactly as it is (our guide and the 404 page). It is fast, cheap and very hard to break or attack.
- **Dynamic** means a page built for each visit from stored data (our message board). You need it when the page depends on data that changes, or on who is asking.

> **PM takeaway:** Use static wherever you can. Add dynamic only where you truly need memory or logic. Every dynamic piece is something that can break, be attacked, or cost money.

4. From laptop to live site



- **Git** keeps a history of every change, like save points you can return to. **GitHub** stores that history online. A *commit* is one save point, a *push* sends it to GitHub, and *main* is the branch that represents the live site.
- You connect GitHub to Cloudflare **once**, by hand, in the dashboard. From then on, every push to main is built and published automatically. This is called **CI/CD** (continuous integration and delivery). The practical meaning: **pushing is publishing**, so you check before you push.
- **The repo describes the whole setup.** wrangler.jsonc says what the site is made of: which folder holds the pages, which database to use, the Worker's name. package.json lists the tools and pins exact versions so every build behaves the same. The only thing not in code is the GitHub-to-Cloudflare link, and the README documents it.
- **When something looks wrong:** check the build result on the commit in GitHub, run the tests on your machine, and if needed undo the change (a *revert*) and push again.

5. A map of the project

File	Its role
public/guide.html public/404.html	The static pages: the guide, and the page shown for addresses that do not exist.
src/index.js	The traffic controller. Routes each request and runs the safety checks.
src/validate.js	The rules: length limits, cleaning of input, the bot timer.
src/html.js	Prints visitor text into pages safely.
src/page.js	The message page and the small script that runs in the visitor's browser.
src/db.js	The database table and the queries.

File	Its role
wrangler.jsonc package.json	The hosting setup, and the list of tools with their exact versions.
test/	The automatic checks.
README.md CLAUDE.md	Instructions for humans, and standing rules for the AI.

6. Safety: assume the worst

Any box people can type into will be abused. Before you ship an input, ask: **“What is the worst thing someone could type here?”** This is what we guarded against.

Threat	In plain words	What we did
SQL injection	Someone types database commands into a form, hoping the database runs them.	Database commands are fixed text in our code. What visitors type is passed separately and is only ever treated as data.
HTML injection (XSS)	Someone types page code, such as a script, hoping every visitor’s browser runs it.	Everything visitors write is turned into harmless text before it appears. A browser-enforced rule (CSP) is a second layer: only our own code may run, proven by a one-time password (a *nonce*) created for each page load.
Bots and spam	Programs that fill in forms automatically.	A hidden field that only bots fill in (a *honeypot*), and a timer: a human needs more than 3 seconds to write a message. Either one rejects the post, saves nothing and keeps the person’s text.
Oversized input	Huge messages meant to slow or fill the system.	Limits: name 40 characters, message 500, request size capped.
Other sites posting for a visitor	A different website secretly submits the form from a visitor’s browser.	The server refuses posts that do not come from our own pages.
Double clicks	The same message saved twice.	The same message sent twice in a short time is stored once.

Safety is layered on purpose: if one layer fails, the next one still protects you.

7. Making it pleasant: progressive enhancement

We built the plain version first: an ordinary form that works with JavaScript switched off. JavaScript then adds the extras: a live character counter, posting without a reload with a thank-you note, and message times in the reader’s own timezone and language. The server cannot know a visitor’s timezone, but their browser can, so the browser does that job.

The benefit is robustness. The page keeps working on old or locked-down devices, and it degrades gracefully instead of breaking.

8. Testing: why it is worth the effort

Kind of test	What it checks	Example
Unit tests	One small rule at a time.	A 501-character message is rejected.
End-to-end tests	The whole running app, used over the network.	Post a message and see it listed newest first.
Real-browser tests	Real Chrome, acting like a visitor.	Post without a reload, see the right local time, hostile text stays harmless.

At the last count there were 67 unit tests and 47 end-to-end tests (114 in all), and all passed. Tests earned their keep. They caught that our own security policy was blocking the page's background request, and a second security error that appeared on every post. Neither would have been obvious to a person reviewing the page. They also caught invisible characters the AI tool had written into the code by accident. The lesson: AI-written code is fast, and it needs checking like any other code.

> **Rule of thumb:** "Done" means the tests pass, the build is green, and a person looked at it.

9. How the work happened, and your role

- You describe the **outcome and the limits**: "visitors leave a message, newest first, use D1, keep the guide at /guide, make it safe."
- The AI proposes, builds, tests, and reports what it could and could not verify.
- Standing rules live in the repo, in a file called CLAUDE.md: the project's memory. Examples: "everything in code, push to main, run the tests first." They apply in every session.
- You review the result in a browser and decide. The product judgement stays with you; the AI does the typing.

10. Next time: the recipe

1. **Write down what visitors do and what must be remembered.** That is your whole spec.
2. **Choose the pieces.** A fixed page: a static file. Logic: a Worker. Remembered data: a database.
3. **Set up the repo and the Cloudflare link once.**
4. **Build the smallest working version.** Push it and watch it go live.
5. **Add safety for every input.** Assume the worst.
6. **Add tests,** and run them before every push.
7. **Add the extras as enhancements,** on top of a version that already works.
8. **Check the live site and keep the docs current.**

A brief you can reuse with the AI

> Build [what] for [who]. Visitors can [actions]. Remember [data]. Host on Cloudflare Workers with static assets, and D1 if data must be stored. Keep all config in code; a push to main deploys. Limit lengths, escape output, add a bot trap. Test locally before pushing, and tell me what you could not verify.

Questions to ask any engineer

- Where is the data stored, and who can see it?
- What happens when someone sends bad input?
- How do we undo a bad release?
- How do we know it works, and how will we notice when it stops working?
- Who can delete content, and how?
- What does it cost if usage grows?

11. Glossary

Term	Meaning
Browser / server	The browser asks; the server answers.
HTML, CSS, JavaScript	Structure, looks, and behavior of a page.
Front end / back end	What people see / the logic behind it.
Serverless	You write the code; the provider runs it on demand. No server to look after.
Worker	Cloudflare’s name for a small program that runs on demand.
Database (D1) / SQL	Organized storage that remembers data between visits / the language used to ask it questions.
Static / dynamic	A file served as is / a page built per visit from data.
Git, commit, push, branch	History of changes / one save point / send changes to GitHub / a line of work; *main* is the live one.
CI/CD, deploy	Automatic building and publishing after each push / publishing a new version.
HTTP status codes	200 OK, 303 redirect, 400 bad request, 403 forbidden, 404 not found, 422 invalid input.
Dependency, lock file	Outside code we rely on / the exact versions, frozen so builds repeat.
Escaping	Turning special characters into harmless text.
CSP, nonce	A browser rule for what code may run / a one-time password for each page load.
Honeypot	A hidden trap field that only bots fill in.
Progressive enhancement	Build the basic version first; layer extras on top.

12. What we did not do, and what to learn next

- There is **no admin or moderation page**, and no per-visitor rate limit. Deleting a message is done by hand with a command.
- The guide’s health advice has **not been reviewed by a clinician.**
- There is no monitoring or analytics, and no custom domain.
- We have not checked **costs at scale.** Look at Cloudflare’s current limits and pricing before promoting the site widely.

Good next things to learn: Cloudflare Turnstile (stronger bot protection), a login-protected admin page for moderation, a custom domain, and basic analytics.